

Module 1: AI Agent Observability and AgentOPs

Demo-1

Demo-1: AI Observability with Langsmith

Scenario

In enterprise AI applications, particularly in HR and recruitment workflows, understanding and monitoring AI system behavior is crucial for maintaining quality, compliance, and performance. The resume screening application demonstrates a real-world scenario where multiple AI operations occur:

- **Document Processing:** Extracting text from various file formats (PDF, DOCX, TXT)
- **Text Analysis:** Using LLMs to analyze resume-job requirement alignment
- **Vector Storage:** Storing analysis results for future reference
- **Score Extraction:** Processing AI responses for structured outputs

Without proper observability, teams face challenges in debugging failures, optimizing performance, and ensuring consistent AI behavior across different inputs and scenarios.

Problem Statement

Traditional AI applications, including the original resume screening system, suffer from several observability gaps:

1. Black Box Operations

- No visibility into LLM reasoning processes
- Inability to trace decision-making steps
- Lack of intermediate state monitoring

2. Performance Monitoring

- No metrics on processing times
- Inability to track resource utilization

- Missing insights into bottlenecks

3. Quality Assurance

- No systematic tracking of AI output quality
- Difficulty in identifying edge cases
- Limited ability to compare different runs

4. Scalability Concerns

- Missing analytics for usage patterns
- Lack of historical performance data

Solution with LangSmith Observability

LangSmith provides comprehensive observability for AI applications through:

Distributed Tracing

Every function call is traced with detailed metadata, enabling complete visibility into the application workflow.

Real-time Monitoring

Live dashboard integration provides immediate insights into application performance and behavior.

Error Tracking

Structured error handling with detailed context for rapid debugging and issue resolution.

Performance Analytics

Detailed metrics on processing times, token usage, and system performance across different operations.



Session Management

Comprehensive tracking of user sessions and analysis patterns for better understanding of usage scenarios.

Project Structure

resume-screening-langsmith/

app.py - # Enhanced Streamlit app with LangSmith integration

.env - # Environment variables (API keys)

requirements.txt - # Dependencies including LangSmith

Core Components with LangSmith Integration:

- **Main Application (app.py):** Enhanced with @traceable decorators and comprehensive observability
- **Vector Storage:** Chroma database with LangSmith tracking for storage operations
- **LLM Chain:** Google Gemini integration with full LangSmith observability
- **Session Management:** UUID-based session tracking with real-time metrics

Setup and Installation

Prerequisites

- Python 3.8+
- Google AI API access
- LangSmith account and API key

Environment Configuration

Create a **.env** file with required API keys:

```
GOOGLE_API_KEY=your_google_api_key_here
```

```
LANGCHAIN_API_KEY=your_langsmith_api_key_here
```

```
LANGCHAIN_PROJECT=resume-screening-demo
```

Dependencies Installation

Create a file **requirements.txt** with the dependencies

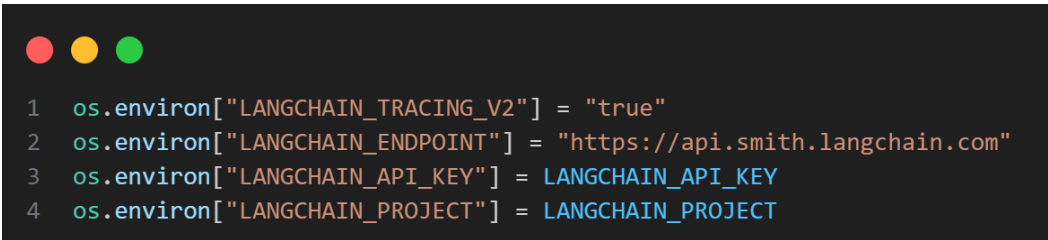


```
1 streamlit==1.29.0
2 langchain==0.1.0
3 langchain-core==0.1.15
4 langchain-google-genai==0.0.5
5 langchain-community==0.0.13
6 google-generativeai==0.3.2
7 python-dotenv==1.0.0
8 docx2txt==0.8
9 pypdf==3.17.1
10 chromadb[all]
```

LangSmith Environment Setup



The application automatically configures LangSmith environment variables:



```
1 os.environ["LANGCHAIN_TRACING_V2"] = "true"
2 os.environ["LANGCHAIN_ENDPOINT"] = "https://api.smith.langchain.com"
3 os.environ["LANGCHAIN_API_KEY"] = LANGCHAIN_API_KEY
4 os.environ["LANGCHAIN_PROJECT"] = LANGCHAIN_PROJECT
```

Setting up LangSmith

API Key Configuration

The application validates LangSmith connectivity at startup:

```
1 # Configure API Keys
2 GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")
3 LANGCHAIN_API_KEY = os.getenv("LANGCHAIN_API_KEY")
4 LANGCHAIN_PROJECT = os.getenv("LANGCHAIN_PROJECT", "resume-screening-demo")
5
6 # Validate required API keys
7 if not GOOGLE_API_KEY:
8     st.error("❌ Please set your GOOGLE_API_KEY in a .env file")
9     st.stop()
10
11 if not LANGCHAIN_API_KEY:
12     st.error("❌ Please set your LANGCHAIN_API_KEY in a .env file for LangSmith integration")
13     st.stop()
```

Client Initialization

```
1 from langsmith import Client, traceable
2 langsmith_client = Client()
3
```

Project Configuration

```
1 LANGCHAIN_PROJECT = os.getenv("LANGCHAIN_PROJECT", "resume-screening-demo")
```

Running the Demo

Create a Virtual Environment:

- Navigate to the project directory.
- Run: `python -m venv venv` (or `env` if preferred).
- Activate the virtual environment:
- Windows: `venv\Scripts\activate`
- macOS/Linux: `source venv/bin/activate`

Install Dependencies:

- Ensure requirements.txt is in the project directory.
- Run: `pip install -r requirements.txt`

Run the Application Locally:

- Run: `streamlit run app.py`

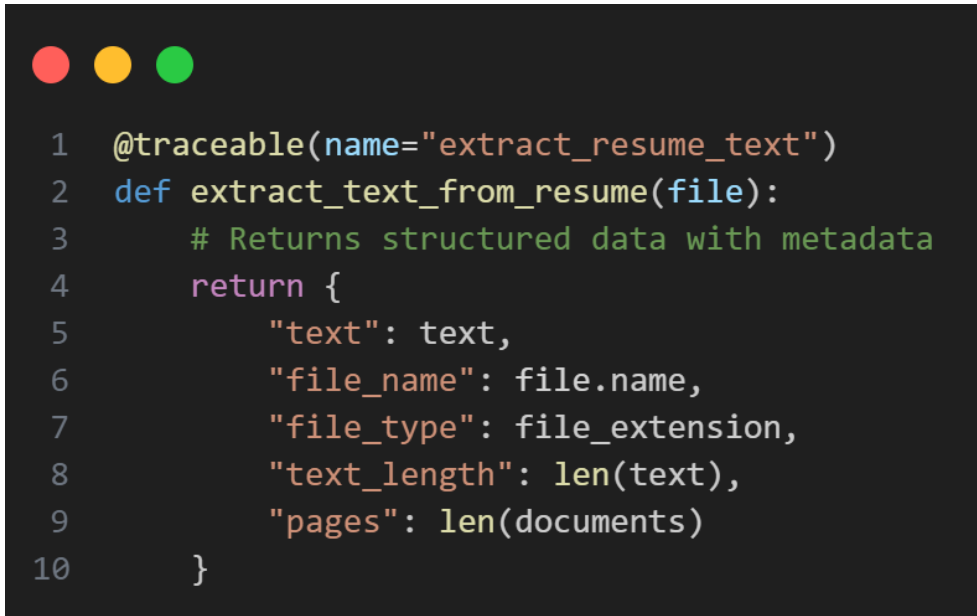
AI Observability with LangSmith

Comprehensive Function Tracing

Every critical function is enhanced with the `@traceable` decorator:

Resume Text Extraction

This function extracts text content from uploaded resume files while capturing detailed metadata about the file processing operation for LangSmith observability.



```
1 @traceable(name="extract_resume_text")
2 def extract_text_from_resume(file):
3     # Returns structured data with metadata
4     return {
5         "text": text,
6         "file_name": file.name,
7         "file_type": file_extension,
8         "text_length": len(text),
9         "pages": len(documents)
10    }
```

Text Chunking with Analytics

This function splits large text into manageable chunks for vector storage while providing analytics on chunking performance and efficiency metrics.

```
1 @traceable(name="split_text")
2 def split_text(text):
3     # Returns analytics alongside processed data
4     return {
5         "documents": documents,
6         "chunk_count": len(documents),
7         "avg_chunk_size": sum(len(doc.page_content) for doc in documents) / len(documents)
8     }
9
```

Vector Storage Operations

This function stores resume analysis results in the vector database while tracking comprehensive metrics about the storage operation and data characteristics.

```
1 @traceable(name="store_resume_analysis")
2 def store_resume_analysis(resume_text, analysis, doc_id):
3     # Comprehensive storage tracking
4     return {
5         "status": "success",
6         "chunks_stored": len(documents),
7         "doc_id": doc_id,
8         "resume_length": len(resume_text),
9         "analysis_length": len(analysis),
10        "avg_chunk_size": split_result["avg_chunk_size"]
11    }
```

End-to-End Analysis Tracking

This function orchestrates the complete resume analysis workflow, coordinating LLM processing while capturing comprehensive metadata about the entire analysis pipeline.


```
1 @traceable(name="analyze_resume_complete")
2 def analyze_resume_with_langsmith(job_requirements, resume_data):
3     # Complete workflow tracking with metadata
4     return {
5         "analysis": analysis,
6         "suitability_score": suitability_score,
7         "file_info": resume_data,
8         "job_requirements_length": len(job_requirements),
9         "analysis_length": len(analysis),
10        "processing_metadata": {
11            "model": "gemini-2.0-flash",
12            "temperature": 0.2,
13            "timestamp": datetime.now().isoformat()
14        }
15    }
16
```

Resume Screening Application

Streamlit Application

The screenshot shows a Streamlit application titled "AI Resume Screening with LangSmith Observability". On the left, a sidebar displays "LangSmith Observability" status as "Connected", the project name "resume-screening-demo", and session details: Session ID "e1b6e943", Timestamp "11:55:19", and "Analyses Completed: 0". The main area has a "Job Requirements" section with a text input containing "Required: Python, Machine Learning, 3+ years experience..." and a red "Analyze Resume" button. To the right is an "Upload Resume" section with a file upload area showing "Resume_Sample 1.pdf" (96.7KB) and a "Deploy" button in the top right corner.

LangSmith Observability

LangSmith Connected

Project: resume-screening-demo

Session ID
e1b6e943

Timestamp
11:55:19

Analyses Completed
0

View Extracted Resume Text

AI Analysis Results

Okay, let's analyze Vaishnavi's resume against the provided job requirements.

1. Skills Match:

- Python:** The resume explicitly lists Python under "Skills Summary" and mentions a "Parking Management System" project using Python. This is a positive match.
- Machine Learning:** While the resume doesn't explicitly list "Machine Learning" under the skills summary, the "Introduction of Artificial Intelligence" certification and the "N-Queen Visualizer" project (potentially involving algorithms) suggest some exposure to related concepts. However, the depth of knowledge is unclear.
- Other Skills:** The candidate also possesses skills in C++, Java, HTML, CSS, JavaScript, MySQL, and DBMS, which could be beneficial depending on the specific role requirements beyond the core skills.

2. Experience Relevance:

- Python Project:** The "Parking Management System" project is the most relevant experience. It demonstrates the ability to design and implement a Python-based application with a user interface. However, the description is brief, and it's difficult to gauge the complexity and scope of the project.
- Other Projects:** The other projects (Hotel Management, N-Queen Visualizer, Hostel Management System) showcase programming skills and experience with different technologies, but they are not directly related to Machine Learning.
- Years of Experience:** This is a critical area where the resume is lacking. The resume does not explicitly state the candidate's work experience. The projects are dated between 2020 and 2022, and the candidate is still pursuing a Bachelor's degree (expected graduation after August 2021). This suggests that the candidate may not have the required 3+ years of professional experience.

LangSmith Observability

LangSmith Connected

Project: resume-screening-demo

Session ID
e1b6e943

Timestamp
11:55:19

Analyses Completed
0

Suitability Score: 45%

Suitability Assessment

Suitability Score
45%
↓ -5% vs baseline

Moderate Match Score: 45%

Analysis stored in vector database and tracked in LangSmith

Storage Metrics

Download Analysis

[View Detailed Traces](#)

Analysis complete! Check LangSmith dashboard for detailed observability traces.

LangSmith Platform Observabilities

1. Project Selection

Navigate to your project: **resume-screening-demo**

Tracing Projects
Analyze and debug applications.

Search by name... Columns

Name	Most Recent Run (7D)	Feedback (7D)	Run Count (7D)	Error Rate (7D)	P50 Latency (7D)	P99 Latency (7D)
resume-screening-demo	5/28/2025, 11:55:26 AM		10	30%	1.27s	5.71s
code_review_demo	5/26/2025, 6:04:38 PM		83	42%	5.12s	9.58s
evaluators	5/25/2025, 10:52:24 PM		6	100%	0.00s	0.02s
tech_solution_observability_de...	5/25/2025, 10:52:01 PM		5	40%	8.46s	10.60s
pr-memorable-stopwatch-14			0	0%		
default			0	0%		

Current sorting (Most Recent Run) excludes projects with no runs. [Sort by Name](#) to see all.

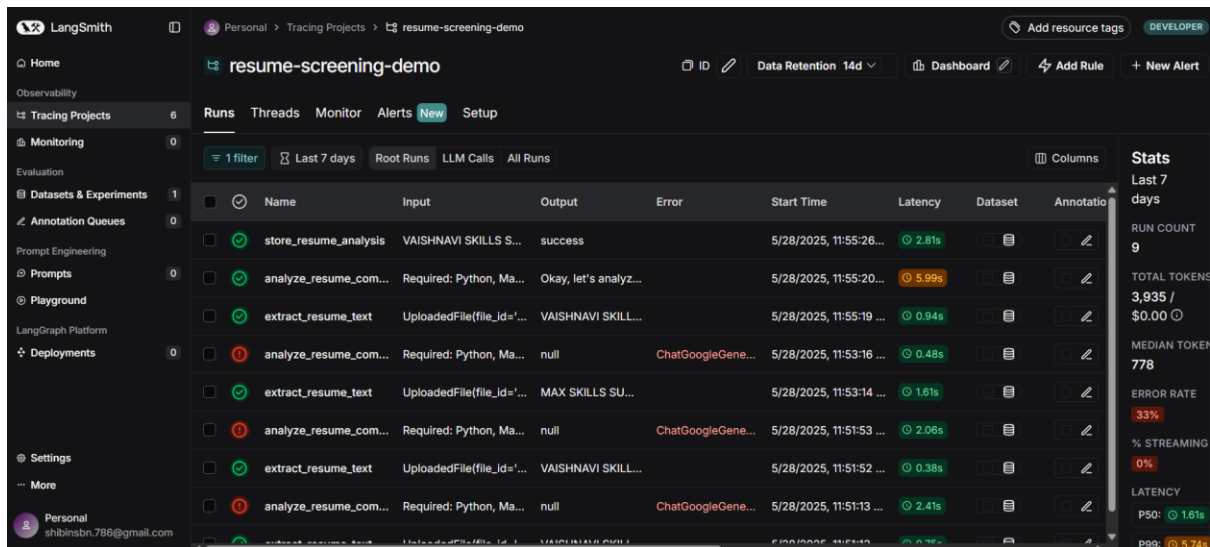
Page 1 < > Show 10

2. Trace Overview Page

Each test case (from your script) generates a new trace. On this page, you'll find:

1. Trace Name
2. Input
3. Output
4. Error
5. Start Time
6. Latency
7. Dataset
8. Annotation Query



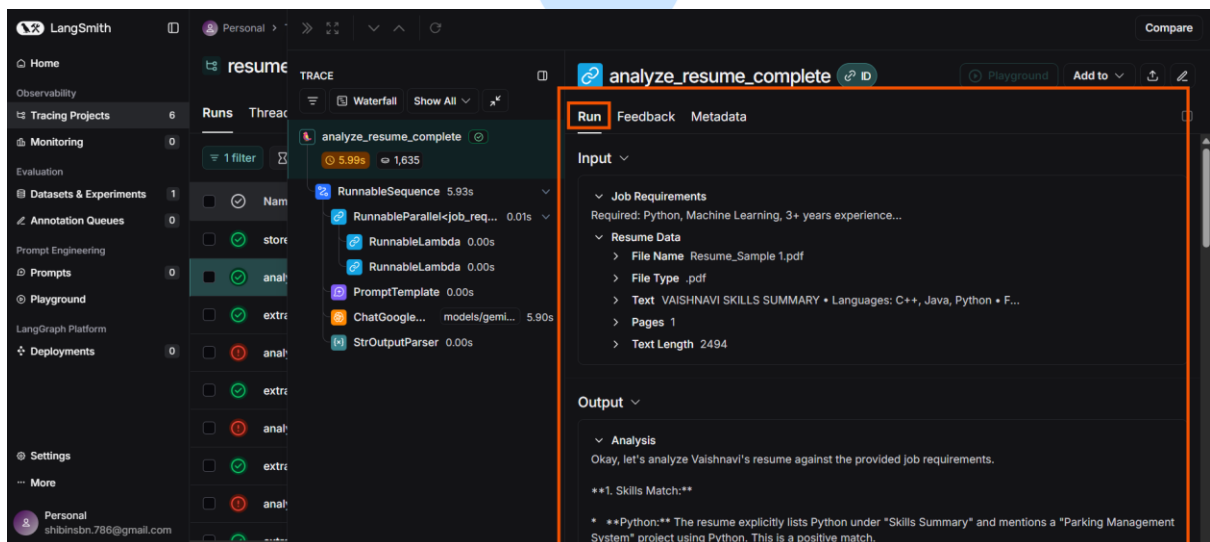


Name	Input	Output	Error	Start Time	Latency	Dataset	Annotations
store_resume_analysis	VAISHNAVI SKILLS S...	success		5/28/2025, 11:55:26...	2.81s		
analyze_resume_com...	Required: Python, Ma...	Okay, let's analyz...		5/28/2025, 11:55:20...	5.99s		
extract_resume_text	UploadedFile(file_id='...	VAISHNAVI SKILL...		5/28/2025, 11:55:19 ...	0.94s		
analyze_resume_com...	Required: Python, Ma...	null	ChatGoogleGene...	5/28/2025, 11:53:16 ...	0.48s		
extract_resume_text	UploadedFile(file_id='...	MAX SKILLS SU...		5/28/2025, 11:53:14 ...	1.61s		
analyze_resume_com...	Required: Python, Ma...	null	ChatGoogleGene...	5/28/2025, 11:51:53 ...	2.06s		
extract_resume_text	UploadedFile(file_id='...	VAISHNAVI SKILL...		5/28/2025, 11:51:52 ...	0.38s		
analyze_resume_com...	Required: Python, Ma...	null	ChatGoogleGene...	5/28/2025, 11:51:13 ...	2.41s		

This will help to find out which all test cases are succeeded and which all test cases are failed

3. Runs in Langsmith

A Run represents a single task or operation carried out within your LLM application, acting as a span that captures one unit of work.



Run	Feedback	Metadata
analyze_resume_complete		

Input

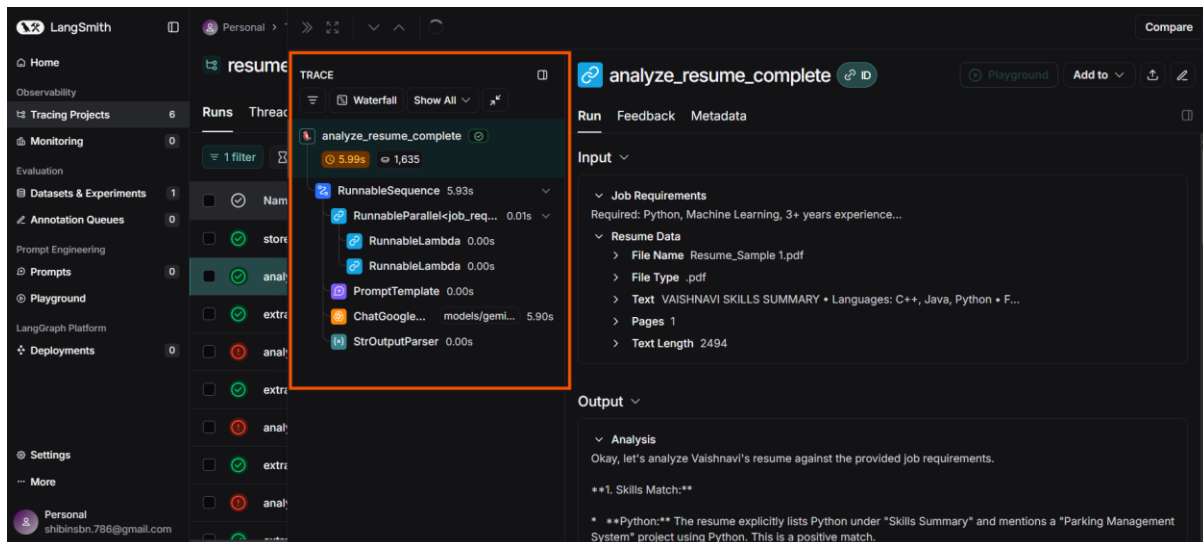
- Job Requirements: Required: Python, Machine Learning, 3+ years experience...
- Resume Data:
 - File Name: Resume_Sample 1.pdf
 - File Type: .pdf
 - Text: VAISHNAVI SKILLS SUMMARY • Languages: C++, Java, Python • F...
 - Pages: 1
 - Text Length: 2494

Output

- Analysis: Okay, let's analyze Vaishnavi's resume against the provided job requirements.
- **1. Skills Match:**
 - * Python: The resume explicitly lists Python under "Skills Summary" and mentions a "Parking Management System" project using Python. This is a positive match.

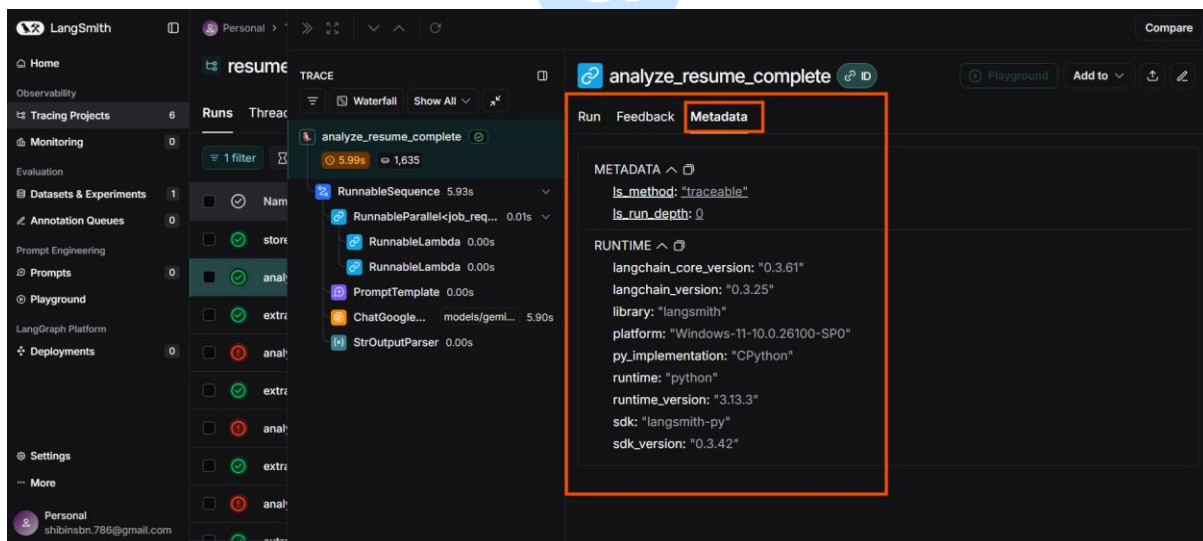
3. Traces in Langsmith

A Trace is a group of related runs that together represent a single end-to-end operation.



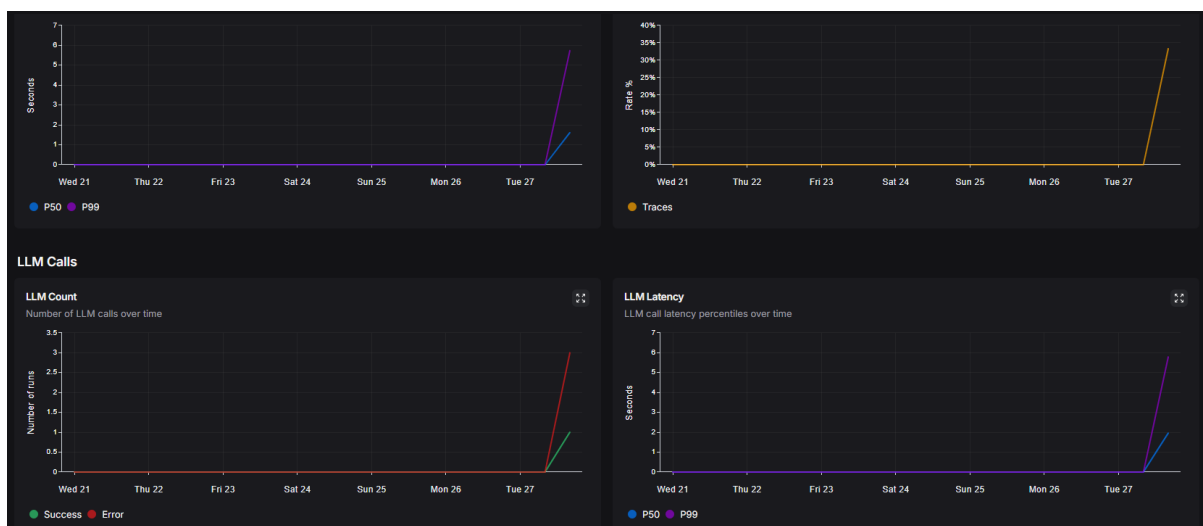
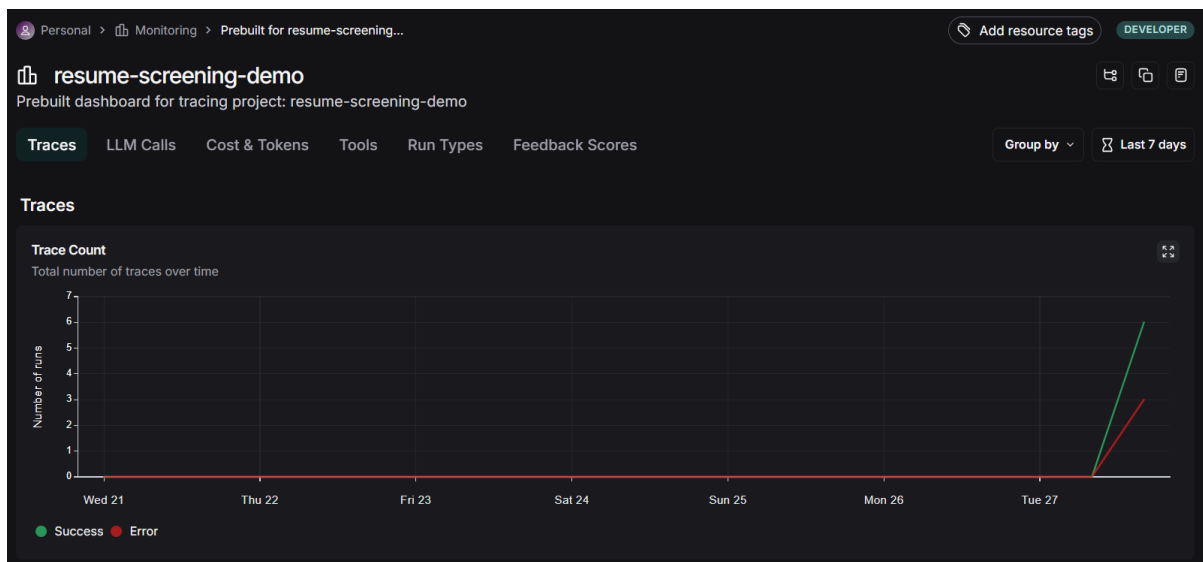
3. Metadata in Langsmith

Metadata consists of key-value pairs that can be linked to runs, allowing you to store extra details - such as the application version, the environment where the run occurred, or any custom information you'd like to associate with that run.



4. LangSmith Monitoring Dashboard

It's tracking and monitoring the performance of an AI application (likely one that does code reviews) over time.



This documentation demonstrates how LangSmith transforms a basic AI application into a fully observable, production-ready system with comprehensive monitoring, debugging, and analytics capabilities.